

Module 5

Topics: More on IF/THEN/ELSE, Converting values between character and numeric

Files to download: [atlgrades.sas7bdat](#)

5.1 – More on IF, ELSE IF and ELSE statements

Remember, when you use IF statements, SAS will evaluate ALL of the data when it processes the line. When you use IF-ELSE statements, the ELSE lines will only process for those records that have not already been dealt with in the previous statement.

```
LIBNAME NICOLE 'H:\AEPI537D';
```

The SAS dataset ATLGRADES includes final course grades for several students in the variable GRADE.

Name	Grade	Name	Grade	Name	Grade	Name	Grade
Ahmad	90	Darius	94	Johnny	87	Tracy	93
Alfred	80	Dave	52	Justin	80	Vanessa	85
Antoine	67	Deshawn	53	Lucas	98	Violet	87
Benny	43	Devynne	91	Michael	85	Willie	95
Bibby	46	Earnest	50	Prescott	75	Zan	60
Chris	86	Franklin	43	Raleigh	90		
Clark	92	Gloria	90	Swift	62		

Let's use a basic IF statement to assign a new variable called COURSE a value of "Pass" if someone has a grade 60 or greater.

```
data ex5_1_a;
    set nicole.atlgrades;
    if grade >= 60 then course = "Pass";
run;
```

You can see from the output here that SAS assigned a value of "Pass" for those with a grade greater than or equal to 60 and left the others blank. Rather than write another statement to specify what to do with values under 60, we could use ELSE to fill in the value of "Fail".

```
data ex5_1_b;
    set nicole.atlgrades;
    if grade >= 60 then course = "Pass";
    else course = "Fail";
run;
```

	Name	Grade	course
1	Ahmad	90	Pass
2	Alfred	80	Pass
3	Antoine	67	Pass
4	Benny	43	
5	Bibby	46	
6	Chris	86	Pass
7	Clark	92	Pass
8	Darius	94	Pass
9	Dave	52	
10	Deshawn	53	
11	Devynne	91	Pass
12	Earnest	50	
13	Franklin	43	
14	Gloria	90	Pass
15	Johnny	87	Pass
16	Justin	80	Pass
17	Lucas	98	Pass
18	Michael	85	Pass
19	Prescott	75	Pass
20	Raleigh	90	Pass
21	Swift	62	Pass
22	Tracy	93	Pass
23	Vanessa	85	Pass
24	Violet	87	Pass
25	Willie	95	Pass
26	Zan	60	Pass

The ELSE statement filled in the “Fail” value on all remaining rows that hadn’t already been assigned a value of “Pass”.

Let’s say we want to assign more than just 2 groups here. We can use IF-ELSE statement to assign letter grades based on a typical scale:

A = 90-100
B = 80-89
C = 70-79
D = 60-69
F = 0-59

Just for fun, look what happens when I try to do this with only IF statements:

```
data ex5_1_c;
  set nicole.atlgrades;
  if grade >= 90 then ltrgrade = "A";
  if grade >= 80 then ltrgrade = "B";
  if grade >= 70 then ltrgrade = "C";
  if grade >= 60 then ltrgrade = "D";
  if grade >= 0 then ltrgrade = "F";
run;
```

Everyone ends up with an F! That’s because each IF statement was processed on the ENTIRE dataset and the condition was always true. So SAS first assigned everyone with a grade >=90 an A, then assigned everyone >=80 a B (which overwrote the As), etc.

To fix this, you could switch up the way you write the IF statements to make sure there will be no overlap.

```
data ex5_1_d;
  set nicole.atlgrades;
  if grade >= 90 then ltrgrade = "A";
  if grade >= 80 & grade < 90 then ltrgrade = "B";
  if grade >= 70 & grade < 80 then ltrgrade = "C";
  if grade >= 60 & grade < 70 then ltrgrade = "D";
  if grade >= 0 & grade < 60 then ltrgrade = "F";
run;
```

Even though this is perfectly fine, you can see how it is still more prone to errors – it would be easy to make a mistake and do something that overlaps. For example, I could accidentally have >=90 on one line, and <=90 on the next – in which case my values =90 would be overwritten on the second line.

	Name	Grade	course
1	Ahmad	90	Pass
2	Alfred	80	Pass
3	Antoine	67	Pass
4	Benny	43	Fail
5	Bibby	46	Fail
6	Chris	86	Pass
7	Clark	92	Pass
8	Darius	94	Pass
9	Dave	52	Fail
10	Deshawn	53	Fail
11	Devynne	91	Pass
12	Earnest	50	Fail
13	Franklin	43	Fail
14	Gloria	90	Pass
15	Johnny	87	Pass
16	Justin	80	Pass
17	Lucas	98	Pass
18	Michael	85	Pass
19	Prescott	75	Pass
20	Raleigh	90	Pass
21	Swiff	62	Pass
22	Tracy	93	Pass
23	Vanessa	85	Pass
24	Violet	87	Pass
25	Willie	95	Pass
26	Zan	60	Pass

	Name	Grade	ltrgrade
1	Ahmad	90	F
2	Alfred	80	F
3	Antoine	67	F
4	Benny	43	F
5	Bibby	46	F
6	Chris	86	F
7	Clark	92	F
8	Darius	94	F
9	Dave	52	F
10	Deshawn	53	F
11	Devynne	91	F
12	Earnest	50	F
13	Franklin	43	F
14	Gloria	90	F
15	Johnny	87	F
16	Justin	80	F
17	Lucas	98	F
18	Michael	85	F
19	Prescott	75	F
20	Raleigh	90	F
21	Swiff	62	F
22	Tracy	93	F
23	Vanessa	85	F
24	Violet	87	F
25	Willie	95	F
26	Zan	60	F

	Name	Grade	ltrgrade
1	Ahmad	90	A
2	Alfred	80	B
3	Antoine	67	D
4	Benny	43	F
5	Bibby	46	F
6	Chris	86	B
7	Clark	92	A
8	Darius	94	A
9	Dave	52	F
10	Deshawn	53	F
11	Devynne	91	A
12	Earnest	50	F
13	Franklin	43	F
14	Gloria	90	A
15	Johnny	87	B
16	Justin	80	B
17	Lucas	98	A
18	Michael	85	B
19	Prescott	75	C
20	Raleigh	90	A
21	Swiff	62	D
22	Tracy	93	A
23	Vanessa	85	B
24	Violet	87	B
25	Willie	95	A
26	Zan	60	D

A more efficient way to do this is using ELSE statements. This code gives us the same results, but is less error prone and we aren't required to know the minimum value for the F category – any rows left by that point are by default values under 60.

```
data ex5_1_e;
  set nicole.atlgrades;
  if grade >= 90 then ltrgrade = "A";
  else if grade >= 80 then ltrgrade = "B";
  else if grade >= 70 then ltrgrade = "C";
  else if grade >= 60 then ltrgrade = "D";
  else ltrgrade = "F";
run;
```

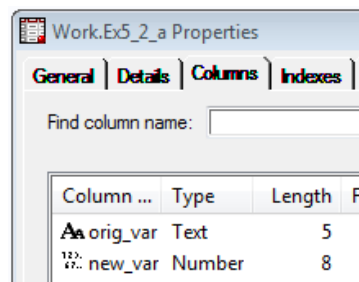
Note that this dataset does not have any missing values, but if it did we would need to handle those as either our first IF statement or a final statement at the end.

5.2 – INPUT to convert values from character to numeric

The following code will produce a character variable called orig_var with a value of "50000" and use the INPUT statement to convert that to a numeric variable called new_var with a value of 50000.

```
data ex5_2_a;
  orig_var = '50000';
  new_var = input(orig_var,8.);
run;
```

Note the variable types in the Properties view for each:



Column ...	Type	Length	Format
orig_var	Text	5	
new_var	Number	8	

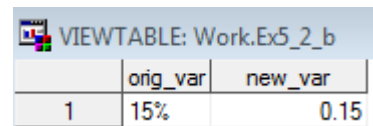
If the goal was to replace the character variable with the numeric one, I could add steps to remove the original character variable and rename the new numeric variable to orig_var.

```
data ex5_2_a;
  orig_var = '50000';
  new_var = input(orig_var,8.);

  drop orig_var;
  rename new_var = orig_var;
run;
```

Here is an example with a percentage that started off as character with the % sign in the field.

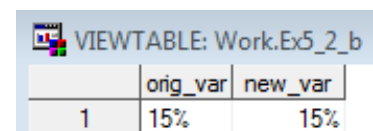
```
data ex5_2_b;
  orig_var = "15%";
  new_var = input(orig_var, percent8.);
run;
```



	orig_var	new_var
1	15%	0.15

We can also keep it numeric but still apply a % format.

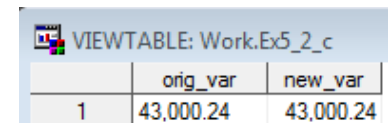
```
data ex5_2_b;
  orig_var = "15%";
  new_var = input(orig_var, percent8.);
  format new_var percent8.;
run;
```



	orig_var	new_var
1	15%	15%

Here is an example with a decimal.

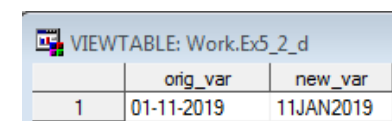
```
data ex5_2_c;
  orig_var = "43,000.24";
  new_var = input(orig_var, comma9.2);
  format new_var comma9.2;
run;
```



	orig_var	new_var
1	43,000.24	43,000.24

Here is an example with a date string converted to a SAS date (and reformatted with another date format).

```
data ex5_2_d;
  orig_var = '01-11-2019';
  new_var = input(orig_var, mmdyy10.);
  format new_var date9.;
run;
```



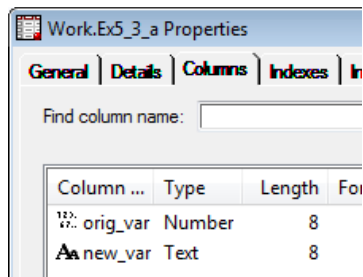
	orig_var	new_var
1	01-11-2019	11JAN2019

5.3 – PUT to convert values from numeric to character

The following code will produce a numeric variable called orig_var with a value of 537 and use the PUT statement to convert that to a character variable called new_var with a value of "537".

```
data ex5_3_a;
  orig_var = 537;
  new_var = put(orig_var, 8.);
run;
```

Note the variable types in the Properties view for each:



Work.Ex5_3_a Properties

General | Details | Columns | Indexes | In

Find column name:

Column ...	Type	Length	For
123 orig_var	Number	8	
AA new_var	Text	8	

Again, if the goal was to replace the numeric variable with the character one, I could add steps to remove the original numeric variable and rename the new character variable to orig_var.

```
data ex5_3_a;

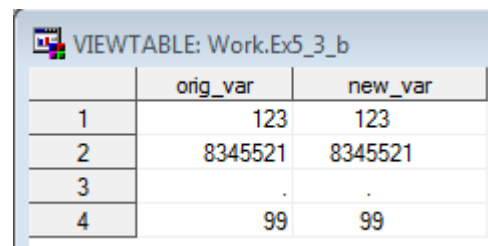
    orig_var = 537;
    new_var = put(orig_var,8.);

    drop orig_var;
    rename new_var = orig_var;

run;
```

Numeric values of different lengths will result in strange alignment.

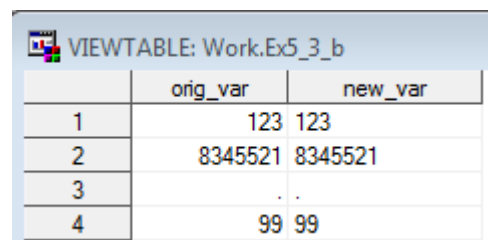
```
data ex5_3_b;
    input orig_var 8.;
    new_var = put(orig_var, 12.);
    datalines;
123
8345521
.
99
;
run;
```



	orig_var	new_var
1	123	123
2	8345521	8345521
3	.	.
4	99	99

You could add a left alignment to clean this up.

```
data ex5_3_b;
    input orig_var 8.;
    new_var = put(orig_var, 12. -L);
    datalines;
123
8345521
.
99
;
run;
```



	orig_var	new_var
1	123	123
2	8345521	8345521
3	.	.
4	99	99

MOD 5: USE SAS COMMENTS TO NUMBER YOUR WORK.

1. Create a new dataset called ATL1 from the dataset ATLGRADES. In this data step, use IF-ELSE statements to create a new character variable called GRADEREV with values of “Excellent” if GRADE is between 85 and 100, “Good” if GRADE is between 70 and 84, “Okay” if GRADE is between 60 and 69, and “Not so good” if GRADE is less than 60. Make sure you are using ELSE statements here!
2. In a dataset called ATL2 made from ATL1, use an INPUT statement to convert the character variable AttendX to a numeric variable called AttendXNum.
3. In a dataset called ATL3 made from ATL2, use a PUT statement to convert the numeric variable Grade to a character variable called GradeChar.
4. In a dataset called ATL4 made from ATL3, use IF-ELSE statements to create a variable called ColorAtd with the following values:
 - ColorAtd = BR0 when
 - Group = “Blue” OR “Red” and AttendXNum = 0
 - ColorAtd = BR1 when
 - Group = “Blue” OR “Red” and AttendXNum = 1
 - ColorAtd = OY0 when
 - Group = “Orange” OR “Yellow” and AttendXNum = 0
 - ColorAtd = OY1 when
 - Group = “Orange” OR “Yellow” and AttendXNum = 1

Look at your dataset to make sure everyone was assigned a proper value!!!